



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Section/Group: 23AIT_KRG-G1

EXPERIMENT- 3

AIM:

Semester: 6th

Student Name: Priyanka

Date of Performance: 28/01/26

UID: 23BAI70303

Subject Name: FULLSTACK-II

Branch: BE CSE AIML

Subject Code: 23CSH-382

To implement centralized state management in the EcoTrack application using Redux Toolkit and to handle asynchronous data operations using Redux async thunks with proper loading and error states.

OBJECTIVES:

After completing this experiment and its follow-up task, the student will be able to:

1. Configure a Redux store in a React application using Redux Toolkit
2. Create and integrate Redux slices for managing application data
3. Implement asynchronous actions using Redux async thunks
4. Manage loading, success, and error states during asynchronous operations
5. Connect React components to Redux state using React-Redux hooks



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

6. Trigger asynchronous data fetching through Redux actions from UI components
7. Use Redux state to derive filtered views without modifying the global store
8. Enhance user experience by handling refresh actions and improving async UI feedback

IMPLEMENTATION/CODE:

logsSlice.js

```
0 | import { createSlice, createAsyncThunk } from
  | '@reduxjs/toolkit';
1 | import initialLogs from '../data/logs'; 2 |
3 | // Simulate an asynchronous API call
4 | export const fetchLogs = createAsyncThunk('logs/fetchLogs', async () => {
5 |   return new Promise((resolve) => {
6 |     setTimeout(() => {
7 |       resolve(initialLogs);
8 |     }, 1500); // 1.5 second delay to show loading state 9 |   });
10| });
11|
12| const logsSlice = createSlice({
13|   name: 'logs',
14|   initialState: {
15|     data: [],
16|     loading: false,
17|     error: null,
18|   },
19|   reducers: {},
20|   extraReducers: (builder) => {
21|     builder
22|       .addCase(fetchLogs.pending, (state) => {
23|         state.loading = true;
24|         state.error = null;
25|       })
26|       .addCase(fetchLogs.fulfilled, (state, action) => {
27|         state.loading = false;
28|         state.data = action.payload;
29|       })
30|       .addCase(fetchLogs.rejected, (state, action) => {
31|         state.loading = false;
32|         state.error = action.error.message;
33|       });
34|   },
35| });
36|
37| export default logsSlice.reducer;
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Store.js

```
0 | import { configureStore } from '@reduxjs/toolkit';
1 | import logsReducer from './logsSlice'; 2 |
3 | const store = configureStore({
4 |   reducer: {
5 |     logs: logsReducer,
6 |   },
7 | });
8 |
9 | export default store;
10| 14|   </div>
15|   );
16| };
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Logs.jsx



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
0 | import { configureStore } from '@reduxjs/toolkit';
1 | import logsReducer from './logsSlice'; 2 |
3 | const store = configureStore({
4 |   reducer: {
5 |     logs: logsReducer,
6 |   },import { useEffect } from "react";
7 | import { useDispatch, useSelector } from "react-redux";
8 | import { fetchLogs } from "../store/logsSlice"; 9 |
10| const Logs = () => {
11|   const dispatch = useDispatch();
12|   const { data, loading, error } = useSelector((state) => state.logs);
13|
14|   useEffect(() => {
15|     dispatch(fetchLogs());
16|   }, [dispatch]);
17|
18|   if (loading) {
19|     return (
20|       <div className="container">
21|         <p>Fetching Eco Logs...</p>
22|       </div>
23|     );
24|   }
25|
26|   if (error) {
27|     return (
28|       <div className="container" style={{ textAlign: 'center', marginTop:
'50px' }}>
29|         <h3 style={{ color: "#ff4d4d" }}>Error</h3>
30|         <p>{error}</p>
31|         <button
32|           style={{ padding: '8px 16px', cursor: 'pointer' }}
33|           onClick={() => dispatch(fetchLogs())}
34|         >
35|           Retry
36|         </button>
37|       </div>
38|     );
39|   }
40|
41|   return (
42|     <div className="container">
43|       <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center' }}>
44|         <h1 className="page-title">Environmental Logs</h1>
45|         <button
46|           style={{ background: 'none', border: 'none', fontSize:
'1.5rem', cursor: 'pointer' }}
47|           onClick={() => dispatch(fetchLogs())}
48|         >
49|           
50|         </button>
51|       </div>
52|     );

```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

53 | `<div style={{ marginTop: '20px' }}>`



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
54|         {data.length > 0 ? (  
55|             <ul style={{ listStyle: 'none', padding: 0 }}>  
56|                 {data.map((log) => (  
57|                     <li key={log.id} style={{
```




DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
58 |         padding: '15px',
59 |         borderBottom: '1px solid #eee',
60 |
61 |         display: 'flex',
62 |         justify-content: 'space-between'
63 |     }>
64 |     <span>{log.activity}</span>
65 |     <span style={{ fontWeight: 'bold' }}>{log.carbon} kg CO2</span>
66 | </li>
67 |     )}
68 | </ul>
69 | ) : (
70 |     <p>No logs found.</p>
71 | )}
72 | </div>
73 | );
74 | };
75 |
76 | export default Logs;
77 | });
78 |
79 | export default store;
```

OUTPUT:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

1)Logs

ECO TRACK	
Summary	Analytics
Logs	Logout
ENVIRONMENTAL LOGS	
Car Travel	4 kg CO ₂
Electricity Usage	6 kg CO ₂
Cycling	0 kg CO ₂

LEARNING OUTCOMES:

1. Implement centralized state management in a React application using Redux Toolkit.
2. Handle asynchronous data operations using createAsyncThunk.
3. Manage loading, success, and error states effectively in Redux.
4. Integrate React components with Redux using React-Redux hooks.
5. Build state-driven, responsive UI updates based on global application state.